

Code of the Day – Power of Atomic Variables

```
std::string* ptr = 0;
int data = 0;
void producer() {
    std::string* p = new std::string("Hello");
    data = 42;
    ptr = p;
}
void consumer() {
    std::string* p2;
    while (!(p2 = ptr)) ;
    assert(data == 42); // may fail due to reordering
}
int main() {
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join(); t2.join();
}
```

```
std::atomic<std::string*> ptr = 0;
int data = 0;
void producer() {
    std::string* p = new std::string("Hello");
    data = 42;
    ptr.store(p, std::memory_order_release);
}
void consumer() {
    std::string* p2;
    while (!(p2 = ptr.load(std::memory_order_acquire))) ;
    assert(data == 42); // never fail
}
int main() { // parallelly run prod/cons
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join(); t2.join();
}
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

OpenMP Tasking

02/24/2025

Before we get started...

- Quick overview, things discussed last time
 - Parallel-for construct to parallelize independent iterations
 - Loop scheduling algorithms used in parallel-for
 - Static: pre-assign the partition to each thread in a round-robin fashion
 - Dynamic: grab a partition whenever a thread is ready for the next round
 - Guided: start with large partition (coarse-grained) and gradually reduce the partition size (fine-grained)
- Purpose of today's lecture
 - Work sharing - sections and tasks
 - Scope of a variable and synchronization
- Miscellaneous
 - Assignment #3 is released – due on **Monday 3/3 at 23:50 PM**
 - Next Monday 3/3 TA will come to discuss assignment #4 and final project details (proposal due on 3/24)

Work Plan, OpenMP

Parallel Regions

Work sharing – Parallel Sections

Variable Scoping Issues

Synchronization

Performance Issues

Loop-based Parallelism

- So far, we have been talking about loop-based parallelism using `omp parallel for`
 - Suitable for independent iterations
 - Suitable for embarrassing parallelism
 - Suitable for regular parallel patterns
- Apply different scheduling algorithms:
 - Static: good for balanced workload
 - Dynamic: good for imbalanced workload
 - Guided: good for common workloads
- Limitations:
 - Can't handle irregular parallel patterns
 - Can't handle complex task dependencies

```
#pragma omp parallel for schedule(static, 8)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```

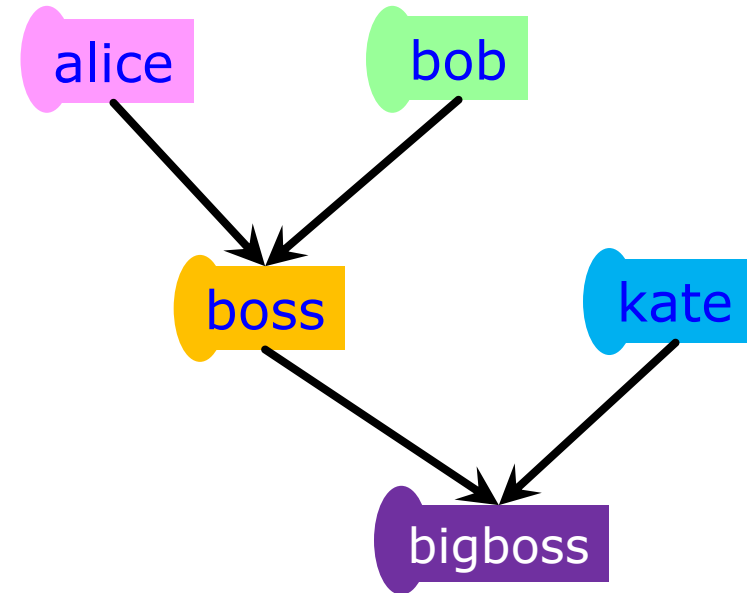
```
#pragma omp parallel for schedule(dynamic, 2)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```

```
#pragma omp parallel for schedule(guided, 1)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```

Function-level Parallelism (aka Task Parallelism)

- Five tasks (functions), with four inter-task dependencies
 - Each task represents an atomic function run by one thread

```
a = alice();  
b = bob();  
s = boss(a, b);  
k = kate();  
std::cout << bigboss(s,k) << '\n';
```



- `alice`, `bob`, and `kate` can be invoked for execution in parallel
- Example above: good illustration of the concept of **task parallelism**

First Solution to Achieve Task Parallelism: **omp sections**

- Allows you to define multiple, independent code blocks (or sections) that can be executed concurrently by different threads within a parallel region

- **#pragma omp sections**

There is an “s” here

- Must be inside a parallel region
- Contains a structured block of N sections of code that may be executed concurrently by T threads
- A barrier is implicitly inserted at the end of the sections construct to ensure all threads complete their sections before proceeding

- **#pragma omp section**

There is **no** “s” here

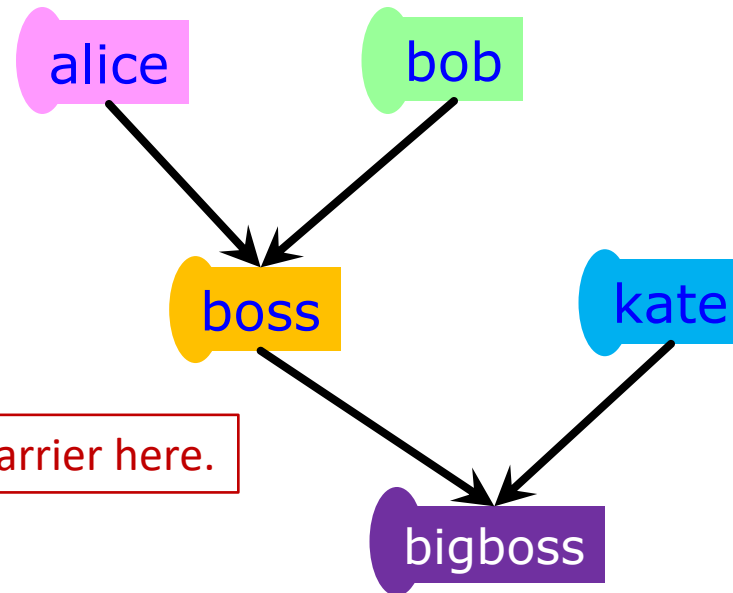
- Contains each section of code within the structured block of “omp sections”
- Each section is an independent task that can run in parallel with other section tasks
- The number of sections is fixed at compile time
- The order in which each section is executed by which threads is not guaranteed or dynamic

Functional Level Parallelism using `omp sections`

```
#pragma omp parallel sections
{
  #pragma omp section
    a = alice();
  #pragma omp section
    b = bob();
  #pragma omp section
    k = kate();
}

double s = boss(a, b);
printf ("%6.2f\n", bigboss(s,k));
```

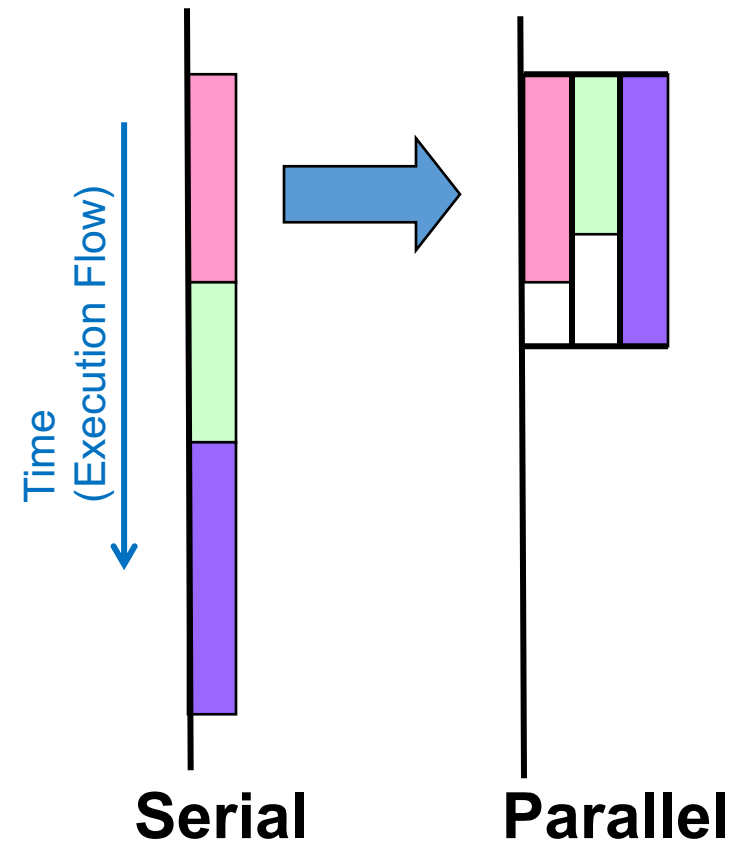
There is an implicit barrier here.



Advantage of **omp sections**

- Independent sections of code can execute concurrently → reduces execution time

```
#pragma omp parallel sections
{
    #pragma omp section
    pink();
    #pragma omp section
    green();
    #pragma omp section
    blue();
}
```



The pink and green tasks are executed at no additional time-penalty in the shadow of the blue task (i.e., they overlap with each other)

Example, **sections**: using 2 threads

```
#include <iostream>
#include <omp.h>
using namespace std;
int main(){
    cout << "\nUsing 2 threads on 3 sections\n";
    double start_time = omp_get_wtime();
    #pragma omp parallel sections num_threads(2)
    {
        #pragma omp section
        {
            printf("Start work 1\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 1\n");
        }
        #pragma omp section
        {
            printf("Start work 2\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 2\n");
        }
        #pragma omp section
        {
            printf("Start work 3\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 3\n");
        }
    }
    cout << "Wall clock time: " << omp_get_wtime() - start_time << endl;
    return 0;
}
```

Each section performs a task that takes 2s to finish

```
$ g++ -std=c++17 test.cpp -fopenmp
$ ./a.out
```

Using 2 threads on 3 sections

Start work 1

Start work 2

End work 1

Start work 3

End work 2

End work 3

Wall clock time: 4.00028

Example, **sections**: using 4 threads

```
#include <iostream>
#include <omp.h>
using namespace std;
int main(){
    cout << "\nUsing 2 threads on 3 sections\n";
    double start_time = omp_get_wtime();
    #pragma omp parallel sections num_threads(4)
    {
        #pragma omp section
        {
            printf("Start work 1\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 1\n");
        }
        #pragma omp section
        {
            printf("Start work 2\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 2\n");
        }
        #pragma omp section
        {
            printf("Start work 3\n");
            std::this_thread::sleep_for(std::chrono::seconds(2));
            printf("End work 3\n");
        }
    }
    cout << "Wall clock time: " << omp_get_wtime() - start_time << endl;
    return 0;
}
```

Each section performs a task that takes 2s to finish

```
$ g++ -std=c++17 test.cpp -fopenmp
$ ./a.out
```

Using 4 threads on 3 sections

Start work 1

Start work 2

End work 1

Start work 3

End work 2

End work 3

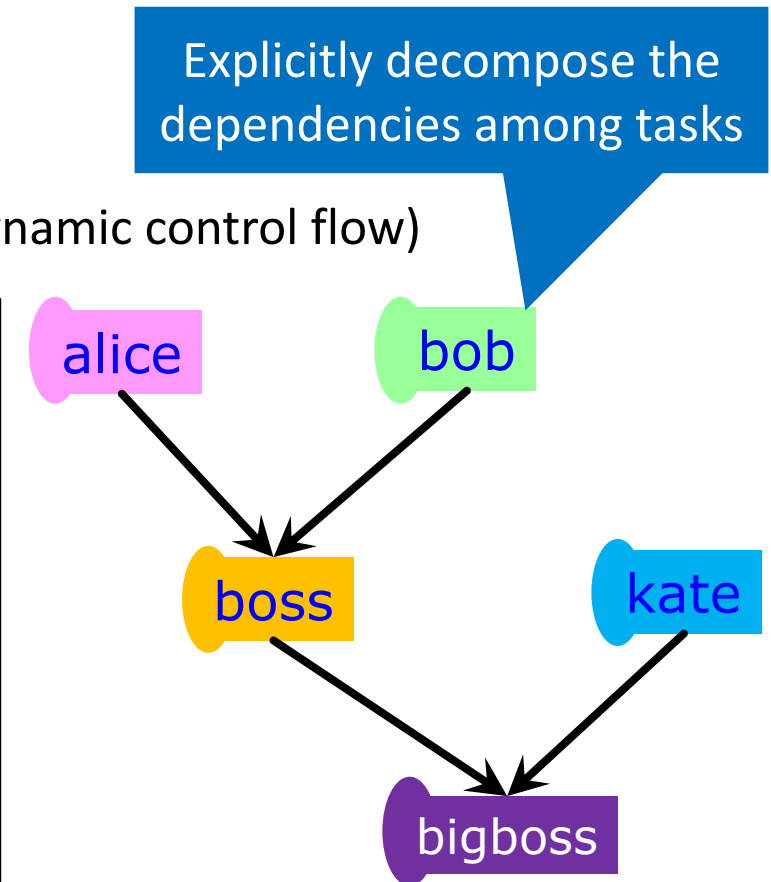
Wall clock time: 2.00028

Disadvantage of **omp sections**

- Manual decomposition
 - Developers need to manually split the tasks into sections, which may not always be feasible for dynamic or highly complex workloads whose dependency constraints depend on runtime variables
- Sections are defined at compile time or programming time
 - Not all applications have known dependencies at compile time (e.g., dynamic control flow)

- Synchronization complexity
 - Section-specific synchronization
 - Ex: boss doesn't overlap with kate

```
#pragma omp parallel sections
{
  #pragma omp section
    a = alice();
  #pragma omp section
    b = bob();
  #pragma omp section
    k = kate();
}
double s = boss(a, b);
bigboss(s, k)
```



Work Plan, OpenMP

Parallel Regions

Work sharing – Parallel tasks

Variable Scoping Issues

Synchronization

Performance Issues

Second Solution to Achieve Task Parallelism: `omp task`

- Define a task using OpenMP task construct: `#pragma omp task`
 - A task is a unit of work that can be executed independently and potentially in parallel with other tasks

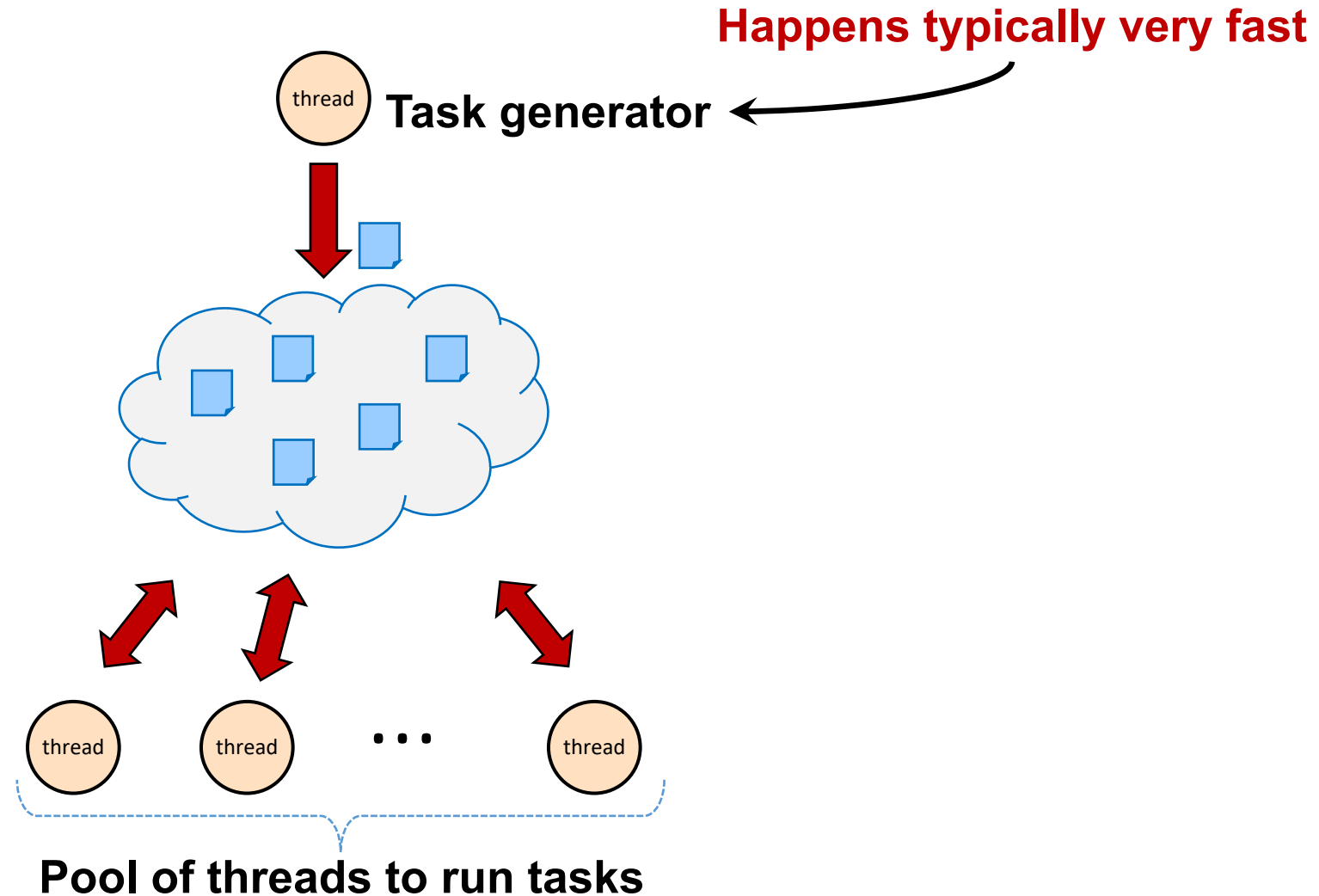
The structured block of code here
is identified an OpenMP “*task*”



```
#pragma omp task
{
    blahblah code
    ...
    smore blahblah code
}
```

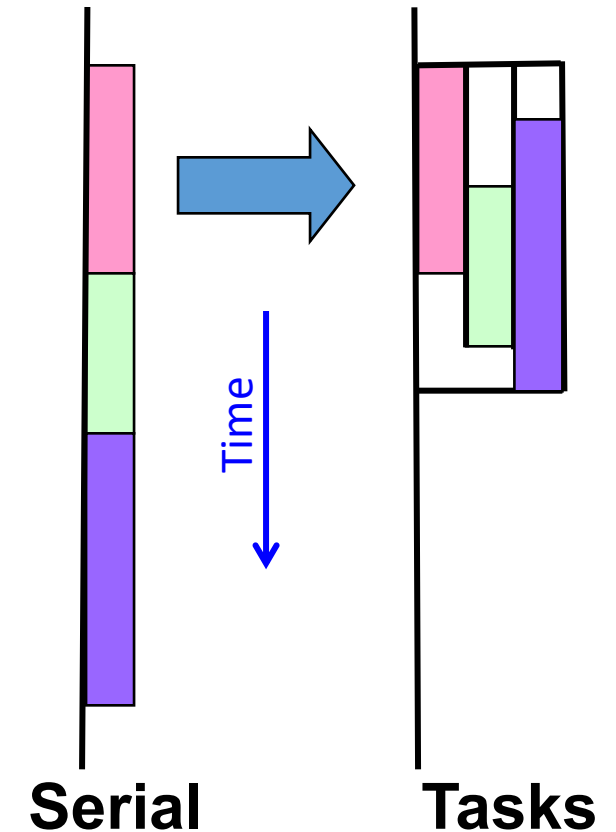
- Whatever is in curly brackets (structured block) is packaged as a “task” that needs to be executed at some point by an OpenMP thread
- Compared with sections, OpenMP task has more flexibility to improve asynchrony
 - Dependency clauses, wait clauses, synchronization clauses, etc.

The **task** Concept in OpenMP: Producer-Consumer Pattern



How does the OpenMP Task Runtime Work?

- Tasks are independent units of work
- One or more threads assigned to generate/post tasks
 - This is called the “**producer**” thread
- The OpenMP threads pick up & execute the “posted” tasks
 - This is called the “**consumer**” thread
- Tasks might be executed immediately or might be deferred
 - Decision is up to the operating system and the OpenMP runtime
 - Applications do not have much control over the scheduling



OpenMP **task**: three important concepts you should understand

- Three important concepts about `#pragma omp task`

- **Code to execute**

- The literal code enclosed by {}
 - If no {} is given, the next immediate line will be the task

```
#pragma omp task
{
    blahblah code
    ...
    smore blahblah code
}
```

- **Data environment**

- By default, every data is captured by reference (shared) rather than copy

- **Internal control variables (ICV)**

- Thread scheduling and environment variables, typically associated with OpenMP

Pros and Cons: OpenMP Tasks

- Pros: allow parallelization of irregular problems
 - Unbounded loops
 - Recursive algorithms
 - Parallelism patterns that depend on runtime variables
 - Producer-consumer patterns
- Cons: requires more attention to deal with and often incurs non-negligible overhead
 - Creating a task and inserting it to the queue
 - Maintaining dependencies involve atomic operations

Motivating Example of OpenMP Task Parallelism (1/3)

Given a linked list of independent task generated from each node via `process(p)`:

```
p = listhead;  
while (p) {  
    process(p);  
    p = next(p);  
}
```

How can we parallelize the execution of these node tasks (i.e., `process(p)`)?

Motivating Example of OpenMP Task Parallelism (2/3)

- Pre-process: traverse the linked list and store node pointers in an array called `arr` and then apply loop-based parallelism:

```
p = listhead;
count = 0;
while (p != listend) {
    arr [count++] = p;
    p = next(p);
}

#pragma omp parallel for
for (int i = 0; i < count; i++)
    process(arr[i]);
```

- **Overhead** of array construction and pre-traversal – especially for long linked list

Motivating Example of OpenMP Task Parallelism (3/3)

- A team of threads is created at **omp parallel** construct
- A single thread is chosen to execute the while loop – let's call this thread “producer”
- Producer thread runs the **while** loop, creates tasks, and fetches next pointers
- Each time the thread crosses the **omp task** construct it generates a new task and has a thread assigned to it
- Each task run by 1 thread at some point in time

```
#pragma omp parallel
{
    #pragma omp single
    {
        *p = head_of_list;
        while(p != end_of_list) {
            #pragma omp task firstprivate(p)
            {
                process(p);
            }
            p = p->next;
        }
    }
}
```

This is important!!!

- All tasks complete at the barrier at the end of the parallel region's construct
- Each task has its **own stack space** that will be destroyed when the task is completed

Using **nowait** to Avoid Synchronization with **single**

- A team of threads is created at **omp parallel** construct
- A single thread is chosen to execute the while loop – let's call this thread “producer”
- Producer thread runs the **while** loop, creates tasks, and fetches next pointers
- Each time the thread crosses the **omp task** construct it generates a new task and has a thread assigned to it
- Each task run by 1 thread at some point in time

Other threads don't need to synchronize with the “single” thread

```
#pragma omp parallel
{
    #pragma omp single nowait
    {
        *p = head_of_list;
        while(p != end_of_list) {
            #pragma omp task firstprivate(p)
            {
                process(p);
            }
            p = p->next;
        }
    }
}
```

This is important!!!

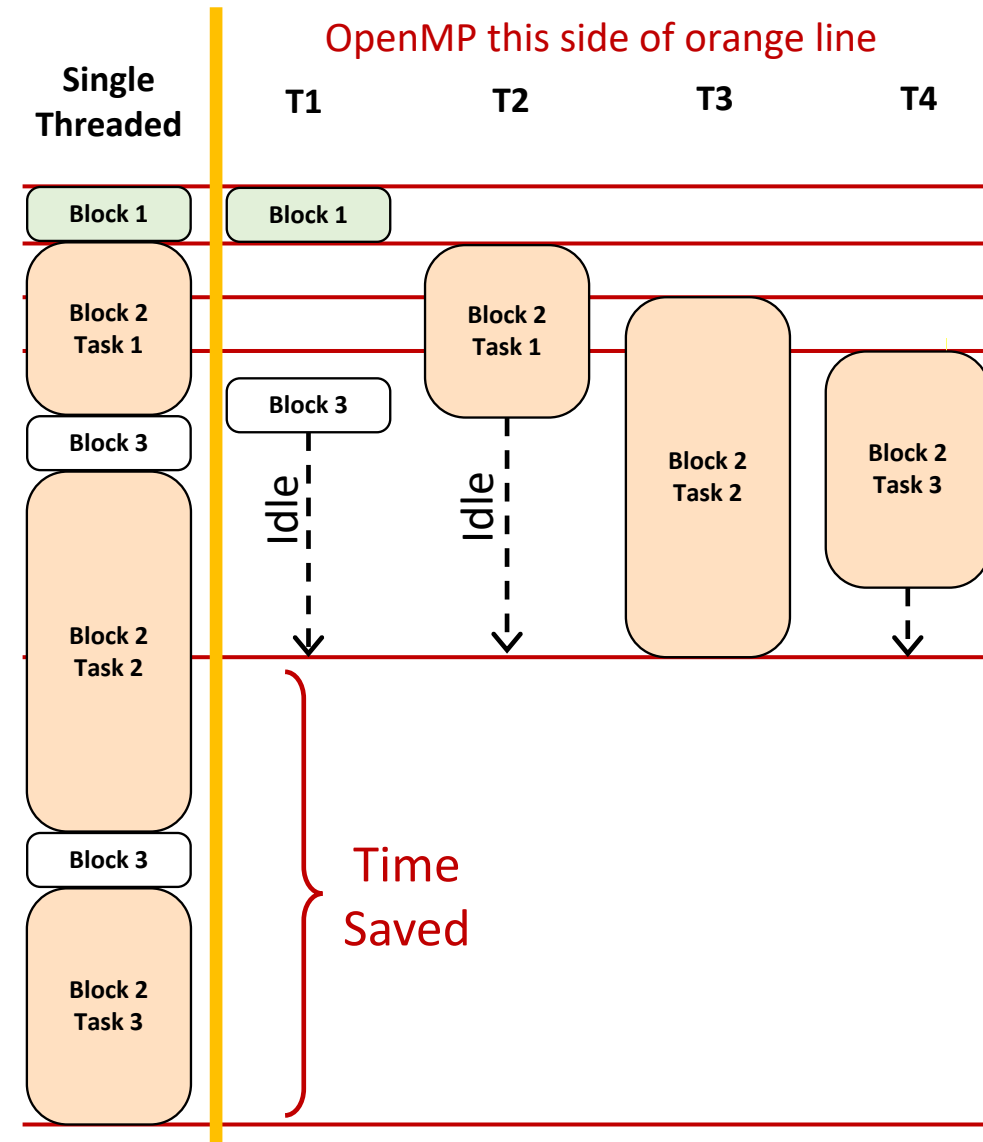
- All tasks complete at the barrier at the end of the parallel region's construct
- Each task has its **own stack space** that will be destroyed when the task is completed

Why Are Tasks Useful?

- Assume 3 big tasks in the linked list
- Assume 4 OpenMP threads
- With **task** you can more easily parallelize irregular and dynamic computational patterns

```
#pragma omp parallel
//threads are ready to go now
{
    #pragma omp single nowait
    {
        // block 1
        *p = head_of_list;
        while(p != end_of_list) {
            #pragma omp task firstprivate(p)
            { //can have a chunk of code here
                process(p); //block 2
            }
            p = p->next; //block 3
        }
    }
}
```

Quick note: `nowait` construct needed since “**single**” calls for implicit synchronization



```

#include <omp.h>
#include <cstdio>
#include <list>
#include <cmath>

void doSomething(std::list<double>::iterator& itr) { *itr *= 2; }

int main() {
    std::list<double> test;
    // initialize the list
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 8; j++)
            test.push_back(std::pow(10.0, i + 1) + j);
    for (auto& item : test) std::printf("%g\n", item);
    auto = test.begin();
    #pragma omp parallel num_threads(8)
    {
        #pragma omp single nowait
        {
            while (it != test.end()) {
                #pragma omp task firstprivate(it)
                {
                    doSomething(it);
                } // end omp task
                it++;
            }
            std::printf("...all tasks posted at this point\n");
        } // end omp single
    } // end omp parallel
    for (auto& item : test) std::printf("%g\n", item);
    return 0;
}

```


Task Creation Pitfall with `omp single` and `omp master`

Recall OpenMP's default behavior:

- Threads **wait** at the end of the `#pragma omp single` block
- Threads **do not wait** at the end of the `#pragma omp master` block

```
#pragma omp parallel
{
    #pragma omp task
    foo();
    #pragma omp barrier
    #pragma omp single
    {
        #pragma omp task
        bar();
    }
}
```

Multiple foo tasks created here – one for each thread

All foo tasks guaranteed to be completed here

One bar task created here

bar task guaranteed to be completed here

Task Dependencies and Synchronization

- Setup:
 - Assume Task B specifically relies on completion of Task A
 - You need to be able to guarantee completion of Task A before invoking the execution of Task B
- **#pragma omp barrier**: tasks are guaranteed to complete at this barrier
 - However, imposing a barrier is an expensive, global effect – *all threads are synchronized at this barrier*
- **#pragma omp taskwait**: child tasks are guaranteed to complete at this wait point
 - When a thread encounters a **taskwait** construct, the current task is suspended until all child tasks generated by this parent thread complete (i.e., per-task synchronization point)
 - Usually, **taskwait** is hit by a thread that operates in an “**omp single**” block; otherwise, you may risk in having **deadlock**
 - Thread of the single block is waiting for other threads to finish while other threads are holding on taskwait

Example of **taskwait**: What Will this Print?

```
#include <omp.h>
#include <iostream>
#include <chrono>
int main()
{
    #pragma omp parallel num_threads(2)
    {
        #pragma omp single nowait
        {
            printf("A ");
            #pragma omp task
            { printf("car "); }
            #pragma omp task
            { printf("race "); }
            #pragma omp taskwait
            printf("is fun to watch.");
        }
    } // End of parallel region
    printf("\n");
    return (0);
}
```

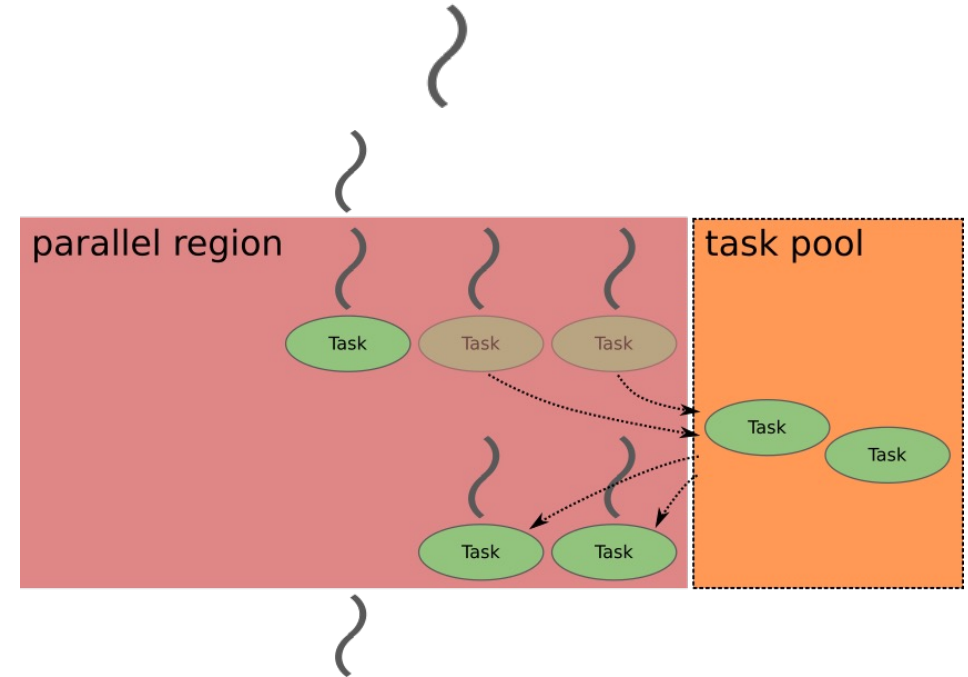
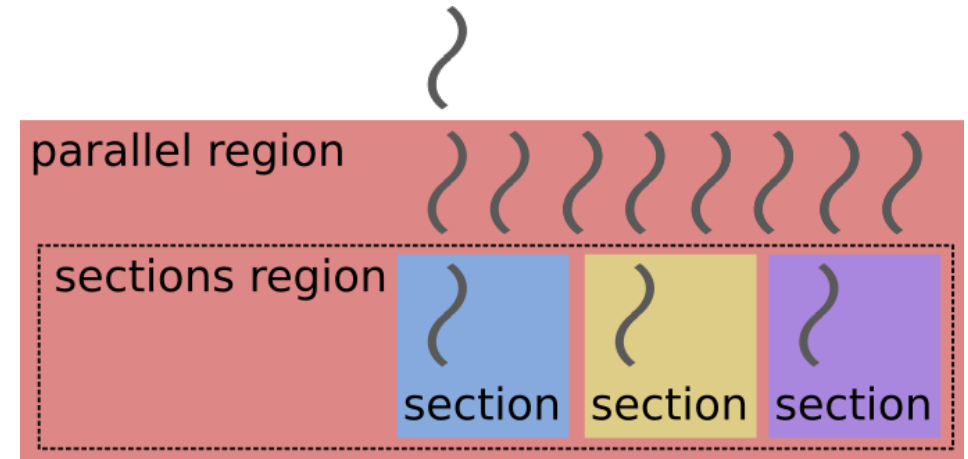
Two possible outputs:

- A car race is fun to watch
- A race car is fun to watch

“is fund to watch” will wait
(i.e., not start) until the previous
two child tasks finish

Takeaway: **section** vs. **task**

- **sections** have a “*static*” attribute
 - Task parallelism is figured out at compile time
- **task** has a “*dynamic*” attribute
 - Task parallelism is figured out at run time
 - A task can encapsulate any block of code
 - Can handle nested loops and dynamic patterns

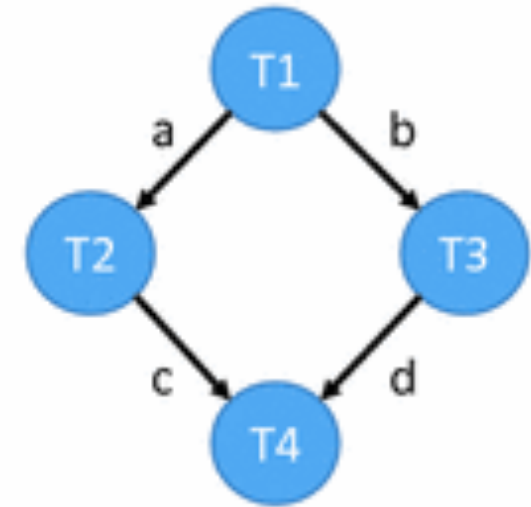


Solution to Task Dependencies: OpenMP Task **depend** Clause

- The **depend** clause allows a more structured way of expressing task dependencies
 - Enable more powerful use of OpenMP tasks in generating dynamic parallelism
- **depend (dependence-type : list)**
 - Dependency type can be **in** or **out** – A dependency $A \rightarrow B$ can be specified with out on A and in on B
 - **List** denote the variable with which the dependence type is associated
- Dependencies are constructed in a topological order to ensure sequential consistency
 - OpenMP tasks are **dynamic** – when you hit a task construct, the runtime will immediately schedule it
 - You need a correct topological order to express the dependency constraints sequentially
 - Why? Think about compiling your OpenMP task code without enabling OpenMP

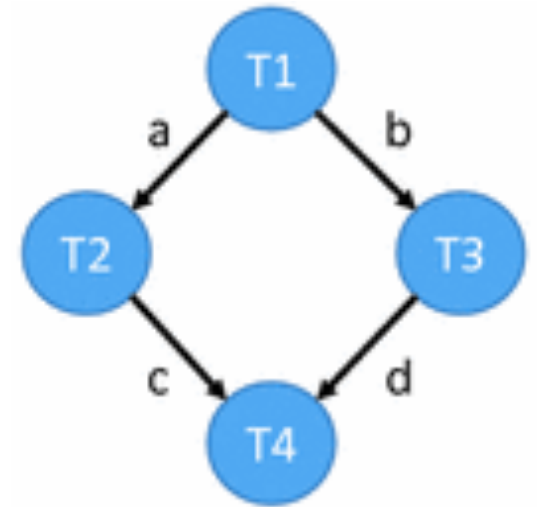
Example of **task depend** Clause

```
#include <omp.h>
int main()
{
    int a, b, c, d;
    #pragma omp parallel
    {
        #pragma omp single nowait
        {
            // T1
            #pragma omp task depend(out: a, b) {}
            // T2
            #pragma omp task depend(in: a) depend(out, c) {}
            // T3
            #pragma omp task depend(in: b) depend(out, d) {}
            // T4
            #pragma omp task depend(in: c, d) {}
        }
    } // End of parallel region
    return (0);
}
```



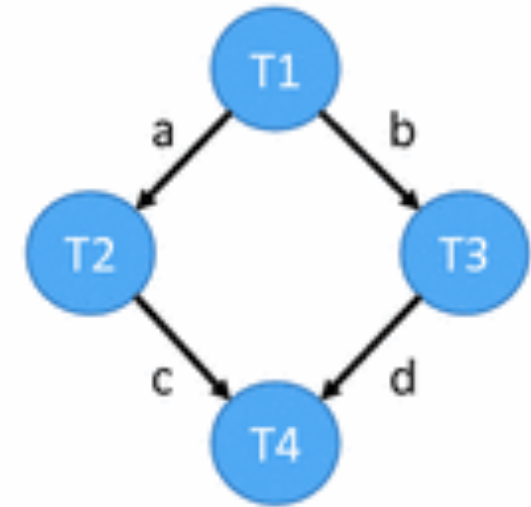
Property of OpenMP Task **depend** Clause

- OpenMP **task depend** clause follows a dynamic task graph parallelism model
 - When a **#pragma omp task** is reached, the runtime immediately spawns a task
 - The dependency constraints can only be related to previously spawned tasks
- Applications must decide a topologically correct order for task creation
 - In our previous example, we can do either
 - $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$, or
 - $T1 \rightarrow T3 \rightarrow T2 \rightarrow T4$
- Why do we need to follow the topological order?
 - When you run the program with OpenMP disabled, everything falls back to sequential execution
 - Only topologically correct order can guarantee correct execution result



Example of Incorrect Use of **task depend** Clause

```
#include <omp.h>
int main()
{
    int a, b, c, d;
    #pragma omp parallel
    {
        #pragma omp single nowait
        {
            // T1
            #pragma omp task depend(out: a, b) {}
            // T4
            #pragma omp task depend(in: c, d) {}
            // T2
            #pragma omp task depend(in: a) depend(out, c) {}
            // T3
            #pragma omp task depend(in: b) depend(out, d) {}
        }
    } // End of parallel region
    return (0);
}
```



Work Plan, OpenMP

Parallel Regions

Work sharing – Parallel tasks

Variable Scoping Issues

Synchronization

Performance Issues

Variable Scoping in OpenMP: Shared and Private Variables

- OpenMP uses a shared-memory programming model
- **Shared variable** - a variable that can be read or written by multiple threads
 - Basic rule: Any variable declared **prior** to a parallel region is shared in that parallel region
 - That is, it is available for read/write by all parallel threads in the parallel region
 - Other examples of variables being shared among threads
 - Global variables
 - File scope variables
 - Namespace scope variables
 - Variables with **const**-qualified type having no **mutable** member
 - **static** variables which are declared in a scope inside the construct
- **Private variable** – a variable whose value is only visible to a thread (for read/write)
 - Basic rule: re-defined variables in the structured block are private to the thread that define those variables

The **private** and **shared** Clauses

- Variable[s] declared **private** in the pragma are reproduced for each thread
 - Consequence: each thread will have a private copy of that variable in the associated parallel region
 - NOTE: Private variables are **un-initialized**
- Variables[s] declared **shared** in the programs are shared by all threads
 - By default, OpenMP captures all variables by reference, sharing captured variables among threads
 - Usually, we use shared only for better readability as by default all variables are shared in OpenMP

```
void work(float* c, int N) {  
    float x, y;  
    int i;  
    #pragma omp parallel for private(x,y)  
    for(i=0; i<N; i++) {  
        x = a[i]; y = b[i];  
        c[i] = x + y;  
    }  
}
```

=

```
void work(float* c, int N) {  
    float x, y;  
    int i;  
    #pragma omp parallel for private(x,y) shared(i)  
    for(i=0; i<N; i++) {  
        x = a[i]; y = b[i];  
        c[i] = x + y;  
    }  
}
```

What's Wrong with this Example?

- Following our previous example ...

```
void work(float* c, int N) {  
    float x, y;  
    int i;  
    #pragma omp parallel for  
    for(i=0; i<N; i++) {  
        x = a[i]; y = b[i];  
        c[i] = x + y;  
    }  
}
```

Default Private Scoping Variable Rules in OpenMP

- Examples of OpenMP variables treated as **private** by default:
 - Stack (local) variables in functions called **from within** parallel regions
 - Loop iteration variables
 - Automatic variables within a statement block

Example [dwelling on the **private** concept]: Is “i” shared or private?

```
#include <omp.h>
#include <cstdio>
using std::printf;

int main() {
    int i = 10;

    #pragma omp parallel num_threads(4) firstprivate(i)
    {
        int threadID = omp_get_thread_num();
        i = 10 + threadID;
        printf("threadID = %d  i = %d\n", me, i);
    }

    printf("i = %d\n", i);

    return 0;
}
```

```
curie ~/ECE759/OpenMP> g++ firstPrivateP0.cpp -fopenmp
curie ~/ECE759/OpenMP> ./a.out
threadID = 0  i = 10
threadID = 2  i = 12
threadID = 1  i = 11
threadID = 3  i = 13
i = 10
curie ~/ECE759/OpenMP> ./a.out
threadID = 3  i = 13
threadID = 0  i = 10
threadID = 2  i = 12
threadID = 1  i = 11
i = 10
curie ~/ECE759/OpenMP> ./a.out
threadID = 0  i = 10
threadID = 3  i = 13
threadID = 1  i = 11
threadID = 2  i = 12
i = 10
curie ~/ECE759/OpenMP> ./a.out
threadID = 0  i = 10
threadID = 3  i = 13
threadID = 2  i = 12
threadID = 1  i = 11
i = 10
curie ~/ECE759/OpenMP>
```

private: Somewhat Prone to Mistake

```
#include <stdio>
#include <omp.h>
using std::printf;
int main()
{
    int i = 10;
    #pragma omp parallel num_threads(4) private(i)
    {
        int threadID = omp_get_thread_num();
        printf("threadID = %d: i = %d\n", me, i);
        i = 1000 + threadID;
    }
    printf("i = %d\n", i);
    return 0;
}
```

- What do we expect each thread to print?
- What do we expect the master thread to print at the end?

```
$ g++ firstPrivateP1.cpp -fopenmp
$ ./a.out
threadID = 0   i = 12345678
threadID = 2   i = -1259487
threadID = 3   i = -8673641145
threadID = 1   i = 6458
i = 10
```

```
$ g++ firstPrivateP1.cpp -Wall -fopenmp
... some stuff here...
firstPrivateP1.cpp:10:11: warning: 'i' is used uninitialized in this function [-Wuninitialized]
    printf("threadID = %d   i = %d\n", threadID, i);
    ~~~~~^~~~~~
firstPrivateP1.cpp:5:7: note: 'i' was declared here
    int i = 10;
    ^
```

Quiz: OpenMP Task Dependency

- What does the following two code snippets print?

```
int x = 100;
#pragma omp parallel {
    #pragma omp single {
        #pragma omp task
        { x *= 2; }
        #pragma omp task
        { x += 1; }
    }
}
printf("x = %d\n", x);
```

```
int x = 100;
#pragma omp parallel {
    #pragma omp single {
        #pragma omp task depend (out: x)
        { x *= 2; }
        #pragma omp task depend (in: x)
        { x += 1; }
    }
}
printf("x = %d\n", x);
```